Ilton Pfleger Júnior

Um Hipervisor Tipo-1 Leve de Tempo Real para Sistemas Embarcados RISC-V: Projeto e Implementação em C++

Florianópolis
2027

Ilton Pfleger Júnior

Um Hipervisor Tipo-1 Leve de Tempo Real para Sistemas Embarcados RISC-V: Projeto e Implementação em C++

Universidade Federal de Santa Catarina

Orientador: Antônio Augusto Fröhlich

Florianópolis
2027

LISTA DE ILUSTRAÇÕES

Figura 1 – Hipervisor Tipo 1. 6
Figura 2 – Hipervisor Tipo 2. 7
Figura 3 – Níveis de Privilégio RISC-V. 9

SUMÁRIO

1	INTRODUÇÃO	4
2	FUNDAMENTAÇÃO TEÓRICA	5
2.1	EPOS	5
2.2	Virtualização	5
2.3	Hipervisors	6
2.3.1	Tipo 1 - Bare Metal	6
2.3.2	Tipo 2	6
2.4	Diferenças Entre Virtualização e Emulação	7
2.5	RISC-V	7
2.5.1	Extensões	8
2.5.2	Níveis de Privilégio	8
2.5.3	*CSRs*	9
2.6	Interceptação e Emulação	9
2.7	*VirtIO*	10
3	METODOLOGIA	11
3.1	Virtualização Baseada em Modo Máquina	11
	REFERÊNCIAS	13

1 INTRODUÇÃO

Sistemas críticos estão presentes em uma ampla gama de aplicações, que vão desde dispositivos médicos e sistemas de automação industrial até plataformas automotivas e missões espaciais. Diferentemente da computação de propósito geral, esses sistemas operam sob rigorosos requisitos de tempo real, nos quais a corretude do comportamento não depende apenas da precisão lógica dos resultados produzidos, mas também do momento exato em que esses resultados são disponibilizados. Em muitos casos, atrasos mínimos ou falhas temporais podem comprometer a segurança, a confiabilidade e até mesmo a integridade física de pessoas e equipamentos.

Tradicionalmente, a garantia desse determinismo esteve associada ao uso de plataformas dedicadas, nas quais Sistemas Operacionais de Tempo Real (RTOSs) são empregados para assegurar previsibilidade temporal, isolamento entre tarefas e cumprimento de restrições rígidas de escalonamento. Entretanto, a evolução do software contemporâneo deslocou grande parte dos ecossistemas de desenvolvimento — bibliotecas, frameworks, ferramentas e serviços — para ambientes baseados em Sistemas Operacionais de Propósito Geral (GPOSs), especialmente Linux. Embora esses sistemas ofereçam grande riqueza funcional e amplo suporte da comunidade, sua adoção em contextos críticos demanda esforços adicionais de adaptação, validação e integração.

Como resultado, cresce a necessidade de consolidar, em uma mesma plataforma computacional, cargas de trabalho com requisitos distintos e frequentemente conflitantes: de um lado, aplicações críticas, sensíveis a prazos temporais e dependentes de comportamento determinístico; de outro, aplicações complexas, orientadas a funcionalidades avançadas e dependentes de ambientes generalistas.

Nesse cenário, técnicas de virtualização surgem como uma alternativa promissora, pois permitem a coexistência de múltiplos ambientes de execução sobre o mesmo hardware. Dessa forma, torna-se possível executar aplicações críticas e não críticas de maneira controlada, reduzindo custos de integração e aumentando o aproveitamento da plataforma.

Paralelamente, a arquitetura aberta RISC-V vem ganhando relevância devido à sua flexibilidade, modularidade e independência tecnológica. Seu conjunto de instruções extensível, aliado à ausência de barreiras proprietárias, favorece tanto a pesquisa acadêmica quanto o desenvolvimento industrial, especialmente em soluções customizadas e de baixo custo.

Por isso, este trabalho propõe o projeto e a implementação, em linguagem C++, de uma base de virtualização para sistemas baseados em RISC-V, com foco em previsibilidade temporal, isolamento entre cargas de trabalho e baixo custo de execução. Busca-se, assim, contribuir para a viabilização de plataformas embarcadas capazes de integrar aplicações heterogêneas sem comprometer requisitos críticos de desempenho e confiabilidade.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 EPOS

Embedded Parallel Operating System (EPOS) Fröhlich e Schröeder-Preikschat (1999) é um RTOS orientado a objetos desenvolvido em C++ com foco em aplicações embarcadas e sistemas dedicados. Sua arquitetura é baseada em componentes modulares, permitindo que apenas os recursos necessários para cada aplicação sejam incluídos no sistema final, reduzindo consumo de memória e overhead.

O EPOS adota o conceito de *Application-Oriented Operating System*, no qual o sistema operacional é configurado de acordo com os requisitos específicos da aplicação utilizando técnicas de orientação a objetos e metaprogramação.

O trabalho desenvolvido neste documento baseia-se nos princípios arquiteturais do EPOS, especialmente em sua modularidade e abstrações.

2.2 VIRTUALIZAÇÃO

A virtualização é uma técnica computacional que permite a execução simultânea e isolada de múltiplos sistemas operacionais em um mesmo hardware físico. Essa abordagem introduz uma camada de abstração entre a máquina hospedeira e os Sistemas Operacionais (SOs) convidados. Por meio dessa abstração, cada SO convidado possui a percepção de acesso exclusivo ao hardware, mesmo que os recursos estejam sendo compartilhados entre diferentes ambientes virtuais.

Historicamente, a virtualização sempre esteve associada ao conceito de hipervisores, que consistem em um arcabouço de software responsável por gerenciar e intermediar o acesso dos sistemas convidados aos recursos físicos da máquina hospedeira.

Na virtualização tradicional, também conhecida como *full virtualization*, o SO convidado executa sem conhecimento explícito de que está em um ambiente virtualizado. Nesse cenário, operações privilegiadas são interceptadas pelo hipervisor e tratadas por mecanismos como *trap-and-emulate*.

Em contraste, a paravirtualização adota uma abordagem cooperativa entre o SO convidado e o hipervisor. Nesse modelo, o sistema convidado é modificado para reconhecer a presença do ambiente virtualizado e, em vez de executar diretamente instruções privilegiadas, realiza chamadas explícitas ao hipervisor, conhecidas como *hypercalls*. Essa estratégia reduz a necessidade de emulação, diminuindo o custo associado à virtualização.

2.3 HIPERVISORS

Um hipervisor, também conhecido como Monitor de Máquina Virtual (VMM), é uma plataforma que cria e executa máquinas virtuais via virtualização. Segundo Popek e Goldberg (1974) um VMM possuí três características essenciais: primeiro, devem fornecer um ambiente idêntico à máquina original para programas; segundo, os programas executados nesse ambiente, na pior das hipóteses, devem apresentar no máximo pequenas reduções de velocidade que urgem da camada de virtualização; e por último, os VMMs devem deter o contole total sobre os recursos do sistema.

Na visão de (GOLDBERG, 1973), fundamentalmente, existem dois tipos de hipervisores: 1 e 2.

2.3.1 Tipo 1 - Bare Metal

Hipervisores de tipo 1, também conhecidos como bare metal, são aqueles que executam diretamente sobre o hardware físico, sem a necessidade de um sistema operacional hospedeiro intermediário. Por operar diretamente sobre o hardware, esse tipo de hipervisor geralmente apresenta melhor desempenho e menor sobrecarga.

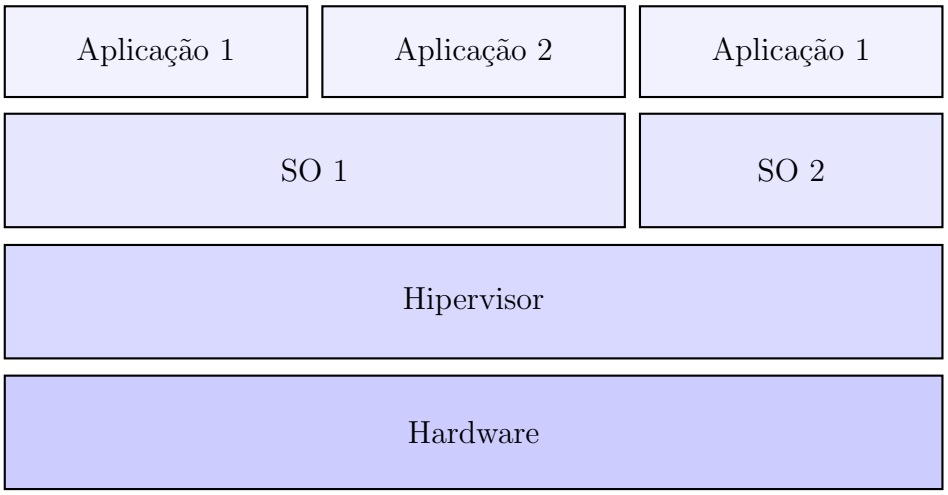


Figura 1 – Hipervisor Tipo 1.

2.3.2 Tipo 2

Diferentemente da solução bare metal, um hipervisor do tipo 2 é executado como um processo dentro de um SO hospedeiro. Porém, apesar de garantir os mesmos recursos do tipo 1, sua arquitetura impõe uma camada intermediária a mais de software, o que restringe a eficiência no acesso aos componentes físicos, gerando uma latência adicional e um menor desempenho.

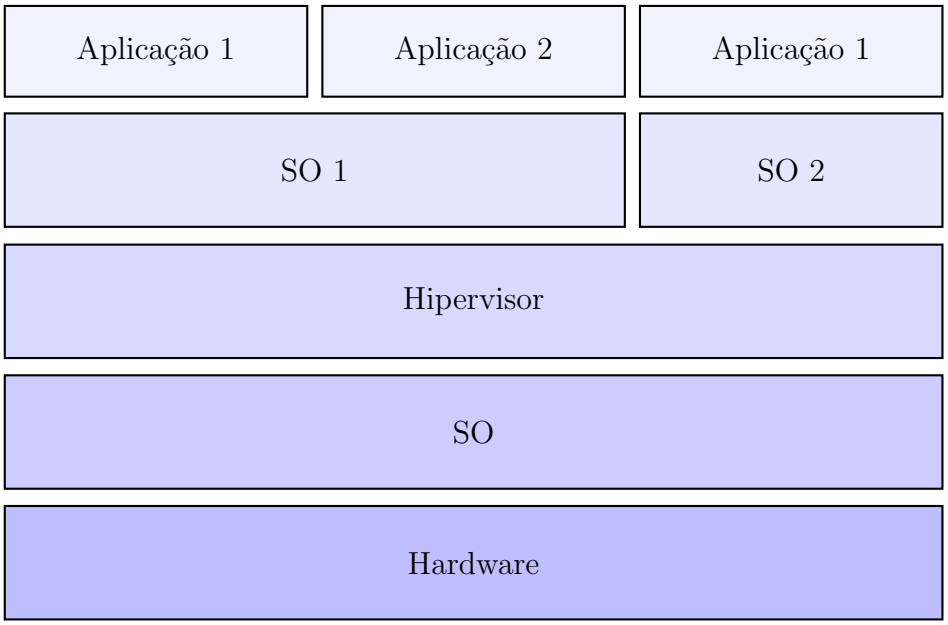


Figura 2 – Hipervisor Tipo 2.

2.4 DIFERENÇAS ENTRE VIRTUALIZAÇÃO E EMULAÇÃO

Algumas funcionalidades utilizadas neste trabalho serão emuladas em vez de virtualizadas, devido a limitações de hardware. A principal diferença entre esses dois conceitos está na forma como as operações são executadas.

A emulação é realizada inteiramente por software, reproduzindo o comportamento de uma arquitetura ou dispositivo por meio da interpretação ou tradução de instruções. Entretanto, devido à necessidade de traduzir instruções e simular componentes de hardware, a emulação tende a apresentar menor desempenho em relação a virtualização.

Por outro lado, embora a virtualização também introduza uma camada de software, esta atua primariamente como uma interface de controle e gerenciamento. Isso permite que a maior parte das instruções seja executada diretamente pelo processador físico. Contudo, essa abordagem exige suporte especializado por parte do hardware subjacente, um requisito que nem sempre está disponível em todas as arquiteturas e máquinas.

2.5 RISC-V

O RISC-V é uma Instruction Set Architecture (ISA) baseada nos princípios da arquitetura RISC (Reduced Instruction Set Computer). Diferentemente de arquiteturas proprietárias, o RISC-V possui uma especificação aberta e livre de royalties, o que permite sua utilização tanto em contextos acadêmicos quanto em aplicações comerciais.

Essa ISA define um conjunto básico de instruções, responsável pelas funcionalidades essenciais do processador. No entanto, sua principal característica é a modularidade, permitindo que diferentes extensões sejam incorporadas conforme as necessidades

da aplicação. Dessa forma, é possível adaptar a arquitetura para cenários variados, desde sistemas embarcados com restrições de recursos até sistemas de alto desempenho.

2.5.1 Extensões

I Adiciona suporte às instruções básicas de operações com números inteiros. Conforme descrito em (INTERNATIONAL, 2024), essa extensão deve estar presente em qualquer implementação RISC-V, pois define as operações fundamentais de aritmética e manipulação de registradores.

Zicsr Adiciona suporte às instruções de acesso e manipulação de Control and Status Registers (CSRs).

H Adiciona suporte em hardware à virtualização. Porém, apesar de existir, é muito recente e sua implementação em hardware comercial ainda é limitada.

M Adiciona suporte à operações de multiplicação e divisão de números inteiros.

A Adiciona suporte à operações atômicas em memória.

F Adiciona suporte a números em ponto flutuante de precisão simples (32 bits).

D Adiciona suporte a números em ponto flutuante de precisão dupla (64 bits).

C Adiciona suporte à instruções comprimidas de 16 bits.

S Adiciona suporte ao modo supervisor.

G Aglomerado geral de extensões que inclui I, M, A, F, D e C.

Para o escopo deste trabalho, serão usadas apenas as extensões Zicsr, S e G. Os motivos e estratégias adotadas para mitigar e justificar a ausência da extensão H serão discutidas ao longo do texto.

2.5.2 Níveis de Privilégio

Assim como a maioria das arquiteturas modernas, a RISC-V utiliza um modelo hierárquico de privilégios para garantir a segurança e a estabilidade dos sistemas que funcionam sobre ela. Esses níveis de privilégio determinam quais operações e recursos de hardware podem ser acessados por cada contexto de execução.

Por exemplo, ao executar o SO em um nível mais privilegiado, torna-se possível protegê-lo contra interferências de aplicações executadas em níveis inferiores. Dessa forma, o sistema operacional pode impedir tanto que aplicações comprometam seu funcionamento quanto que interfiram entre si, aumentando a segurança do sistema como um todo.

Essa proteção é normalmente implementada por meio de mecanismos de hardware, como registradores privilegiados e regiões de memória protegidas, cujo acesso é restrito de acordo com o nível de privilégio de cada contexto de execução.

O modelo básico da arquitetura estudada neste documento, conforme mostra a Figura 3, contém três níveis de privilégio.

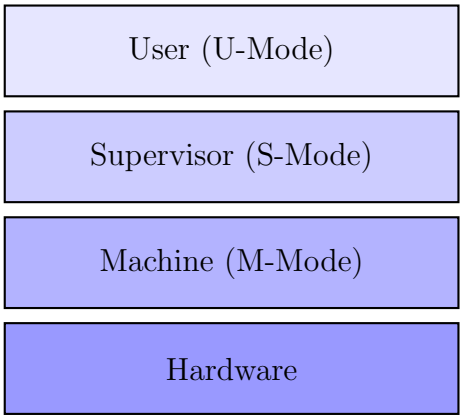


Figura 3 – Níveis de Privilégio RISC-V.

User Mode Nível mais restrito de execução, destinado à operação de aplicações que não devem possuir permissões suficientes para comprometer a integridade, a segurança ou o funcionamento do sistema.

Supervisor Mode Nível utilizado pela maior parte dos SOs modernos, permitindo que eles gerenciem os recursos da máquina e controlem as permissões associadas ao *User Mode*.

Machine Mode Nível mais privilegiado da arquitetura, responsável pelo controle direto do hardware e pela execução de operações críticas de baixo nível. Seus aspectos serão discutidos com maior profundidade ao longo deste texto.

2.5.3 *CSRs*

Os CSRs são registradores especiais utilizados para controlar e monitorar o funcionamento do processador. Esses registradores são responsáveis por armazenar informações relacionadas ao estado da máquina, tratamento de exceções, tratamento de interrupções, configuração de privilégios e controle de funcionalidades específicas da arquitetura.

2.6 INTERCEPTAÇÃO E EMULAÇÃO

Do inglês *trap-and-emulate*, interceptação e emulação trata-se de um método utilizado para viabilizar a virtualização em sistemas nos quais o suporte de hardware não

é completamente implementado. Nesse modelo, quando a aplicação executada em ambiente virtualizado tenta realizar uma operação que depende de recursos privilegiados, essa operação é interceptada e tratada como uma exceção. O controle é então transferido para um nível de privilégio superior, geralmente o hipervisor, que fica responsável por emular em software o comportamento esperado da instrução ou operação originalmente solicitada.

2.7 *VIRTIO*

O *VirtIO* (RUSSELL, 2008) é um modelo de dispositivo virtual projetado para padronizar a comunicação de operações de Entrada e Saída (IO) entre SOs convidados e seus respectivos hipervisores. Seu funcionamento assemelha-se ao de dispositivos físicos baseados em Acesso Direto à Memória (DMA), empregando regiões de memória compartilhada para a transferência direta de dados.

Para viabilizar essa comunicação, o modelo utiliza as chamadas *Virtqueues* — filas circulares compartilhadas responsáveis pelo envio e recebimento das requisições. Dependendo das operações suportadas, cada dispositivo virtual pode alocar uma ou mais dessas filas. Internamente, uma *Virtqueue* divide-se em três componentes principais: uma tabela de descritores (*Descriptor Table*), um anel de índices disponíveis (*Available Ring*) e um anel de índices já utilizados (*Used Ring*).

Na prática, o ciclo de vida de uma operação de IO ocorre em duas etapas complementares. Inicialmente, quando o sistema convidado deseja realizar uma operação, o *driver VirtIO* escreve os dados na memória compartilhada, insere o descritor correspondente no *Available Ring* e notifica o hipervisor sobre a nova pendência.

Em resposta, o hipervisor consome os descritores desse anel, processa a operação solicitada e registra o resultado no *Used Ring*. Por fim, uma interrupção é enviada ao sistema convidado para sinalizar a conclusão da tarefa. Graças a esse design, múltiplas requisições podem ser tratadas de maneira assíncrona e em lote, o que minimiza as trocas de contexto e reduz expressivamente o custo inerente à virtualização.

Desse modo, o *VirtIO* consolida-se como uma solução fundamental para a infraestrutura de virtualização moderna, pois alcança um equilíbrio entre a flexibilidade de uma interface padronizada e o desempenho da paravirtualização.

3 METODOLOGIA

3.1 VIRTUALIZAÇÃO BASEADA EM MODO MÁQUINA

O *Machine Mode* foi concebido originalmente na arquitetura RISC-V como o nível de privilégio mais elevado, responsável pela execução do firmware base e pelo controle direto dos recursos de hardware. O software residente nesse nível encarrega-se da inicialização da plataforma, da configuração dos subsistemas fundamentais e, subsequentemente, da transferência de controle para o SO hospedeiro.

Além das responsabilidades de inicialização, determinadas implementações da arquitetura requerem que o modo máquina forneça serviços privilegiados às camadas superiores. Esses serviços são disponibilizados por meio de interfaces padronizadas, como a Supervisor Binary Interface (SBI), permitindo que sistemas executados em níveis menos privilegiados acessem de forma controlada as funcionalidades de hardware restritas.

No escopo deste trabalho, o M-mode é empregado de forma atípica: como a camada base para a implementação de um hipervisor, visando contornar a ausência da extensão de virtualização (H) na plataforma utilizada. Esta estratégia fundamenta-se na autoridade irrestrita que o modo máquina possui sobre os demais níveis de privilégio da arquitetura, o que viabiliza o controle e a orquestração integral dos SOs convidados. Para tanto, o M-mode dispõe de um conjunto isolado de CSRs e de mecanismos próprios para o tratamento de interrupções e temporização, operando de maneira completamente independente dos modos que operam acima dele.

O isolamento e o controle dos sistemas convidados baseiam-se fortemente no mecanismo de tratamento de exceções da arquitetura. Sempre que o SO virtualizado executa uma instrução privilegiada não permitida ou tenta acessar um recurso restrito, o hardware gera uma exceção que é direcionada nativamente ao modo máquina. Esse comportamento viabiliza a técnica clássica de *trap-and-emulate*, permitindo que o hipervisor intercepte a operação falha e emule o seu comportamento de maneira controlada.

Embora o M-mode careça dos mecanismos de tradução de memória em dois estágios - como a Unidade de Gerenciamento de Memória (MMU) de segundo estágio fornecida pela extensão H -, o isolamento de memória entre os convidados é viabilizado através dos dispositivos de Physical Memory Protection (PMP). Configurado exclusivamente pelo modo máquina, o PMP permite delimitar e proteger regiões específicas da memória física contra acessos indevidos de camadas menos privilegiadas.

Essa granularidade do PMP é também empregada na emulação de *Memory-Mapped I/O* (MMIO), recurso indispensável para o suporte a periféricos virtuais como o *VirtIO*. Esse mecanismo opera mediante a configuração das regiões correspondentes aos dispositivos virtuais como inacessíveis ao sistema convidado; consequentemente, qualquer tentativa

de leitura ou escrita engatilha uma exceção de falha de acesso, permitindo que o hipervisor intercepte a operação e processe a requisição para o SO convidado.

REFERÊNCIAS

FRöHLICH, A.; SCHRöEDER-PREIKSCHAT, W. High performance application-oriented operating systems - the epos approach. In: *Anais do XI Symposium on Computer Architecture and High Performance Computing*. Porto Alegre, RS, Brasil: SBC, 1999. p. 3–9. ISSN 0000-0000. Disponível em: ⟨https://sol.sbc.org.br/index.php/sbac-pad/article/view/19765⟩.

GOLDBERG, R. P. Architectural principles for virtual computer systems. In: . [s.n.], 1973. Disponível em: ⟨https://apps.dtic.mil/sti/tr/pdf/AD0772809.pdf⟩.

INTERNATIONAL, R.-V. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. 2024. ⟨https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf⟩.

POPEK, G. J.; GOLDBERG, R. P. Formal requirements for virtualizable third generation architectures. *Commun. ACM*, Association for Computing Machinery, New York, NY, USA, v. 17, n. 7, p. 412–421, jul. 1974. ISSN 0001-0782. Disponível em: ⟨https://doi.org/10.1145/361011.361073⟩.

RUSSELL, R. virtio: towards a de-facto standard for virtual i/o devices. *SIGOPS Oper. Syst. Rev.*, Association for Computing Machinery, New York, NY, USA, v. 42, n. 5, p. 95–103, jul. 2008. ISSN 0163-5980. Disponível em: ⟨https://doi.org/10.1145/1400097.1400108⟩.